

On this page

Payment Gateway Integration Guide

This guide provides a step-by-step approach to integrate your payment gateway with FluentCart. Follow these steps to create a fully functional payment gateway with support for one-time payments, subscriptions, and web hooks.

Implementation Steps

Step 1: Register Your Gateway

In your plugin's main file, register your gateway with FluentCart using the hook approach (recommended):

```
// In your-plugin.php
add_action('fluent_cart/register_payment_methods', function() {
    if (!function_exists('fluent_cart_api')) {
        return; // FluentCart not active
    }

    // Register your custom gateway
    fluent_cart_api()->registerCustomPaymentMethod(
        'your_gateway',
        new \YourPlugin\PaymentMethods\YourGateway\YourGateway()
    );
});
```

php

Alternatively, you can register on the `init` hook (not recommended):

```
add_action('init', function() {
    if (!function_exists('fluent_cart_api')) {
        return;
    }
    fluent_cart_api()->registerCustomPaymentMethod('your_gateway', new \YourPlu
});
```

Step 2: Create the Settings Class

Create a settings class that extends `BaseGatewaySettings` :

```
<?php
namespace YourPlugin\PaymentMethods\YourGateway;

use FluentCart\App\Modules\PaymentMethods\Core\BaseGatewaySettings;
use FluentCart\App\Helpers\Helper;

class YourGatewaySettings extends BaseGatewaySettings
{
    public $methodHandler = 'fluent_cart_payment_settings_your_gateway';

    public static function getDefaults()
    {
        return [
            'is_active' => 'no',
            'payment_mode' => 'test', // test or live
            'test_api_key' => '',
            'test_secret_key' => '',
            'live_api_key' => '',
            'live_secret_key' => '',
        ];
    }

    public function getApiKey()
    {
        $mode = $this->get('payment_mode');
        return $this->get($mode . '_api_key');
```

```

    }

    public function getSecretKey()
    {
        $mode = $this->get('payment_mode');
        return Helper::decryptKey($this->get($mode . '_secret_key'));
    }

    public function isTestMode()
    {
        return $this->get('payment_mode') === 'test';
    }
}

```

Step 3: Create your Gateway Class

Create base class that implements all required methods:

php

```

<?php
namespace YourPlugin\PaymentMethods\YourGateway;

use FluentCart\App\Modules\PaymentMethods\Core\AbstractPaymentGateway;
use FluentCart\App\Services\Payments\PaymentInstance;
use FluentCart\Framework\Support\Arr;

class YourGateway extends AbstractPaymentGateway
{
    // Define supported features
    public array $supportedFeatures = [
        'payment',
        'webhook',
        'refund',
        'subscriptions'
    ];

    public function __construct()
    {
        // Initialize settings
    }
}

```

```

        parent::__construct(new YourGatewaySettings());
    }

    public function boot()
    {
        // initialize any hanldere, webhook/ payment confirmation class if need
    }

    #required: Return gateway metadata
    public function meta(): array
    {
        return [
            'title' => __('Your Gateway', 'your-plugin'),
            'route' => 'your_gateway',
            'slug' => 'your_gateway',
            'description' => __('Accept payments with Your Gateway', 'your-plug
            'logo' => plugin_dir_url(__FILE__) .
                'assets/images/logo.svg',
            'icon' => plugin_dir_url(__FILE__) .
                'assets/images/icon.svg',
            'status' => $this->settings->get('is_active') === 'yes',
            'supported_features' => $this->supportedFeatures
        ];
    }

    #required: Check if gateway supports a feature
    public function has(string $feature): bool
    {
        return in_array($feature, $this->supportedFeatures);
    }

    // Required: Process payment
    public function makePaymentFromPaymentInstance(PaymentInstance $paymentInst
    {
        // Your payment processing logic here
    }

    // Required: Handle IPNs/Webhooks
    public function handleIPN()

```

```

{
    // Process the webhook
}

#required: Return settings fields configuration
public function fields(): array
{
    // For a comprehensive guide on building gateway settings fields,
    // see the detailed [Payment Gateway Settings Fields] documentation link
    return [
        ....
    ];
}

// For a comprehensive guide on building gateway settings fields,
// see the detailed documentation: [Payment Gateway Settings Fields](./payment-gateway-settings-fields.md)

#required: Get order information for frontend
public function getOrderInfo(array $data)
{
    // Prepare frontend data for checkout
    $paymentArgs = [];

    // Return data for frontend
    wp_send_json([
        'status' => 'success',
        'payment_args' => $paymentArgs,
        'message' => __('Order info retrieved', 'your-plugin')
    ], 200);
}

#required: Register scripts (automatically called by base gateway)
public function getEnqueueScriptSrc($hasSubscription = 'no'): array
{
    // External gateway library, custom checkout scripts (if needed), other
    $gatewayLibUrl = 'https://js.yourgateway.com/v1/checkout.js';

    return [
        [

```

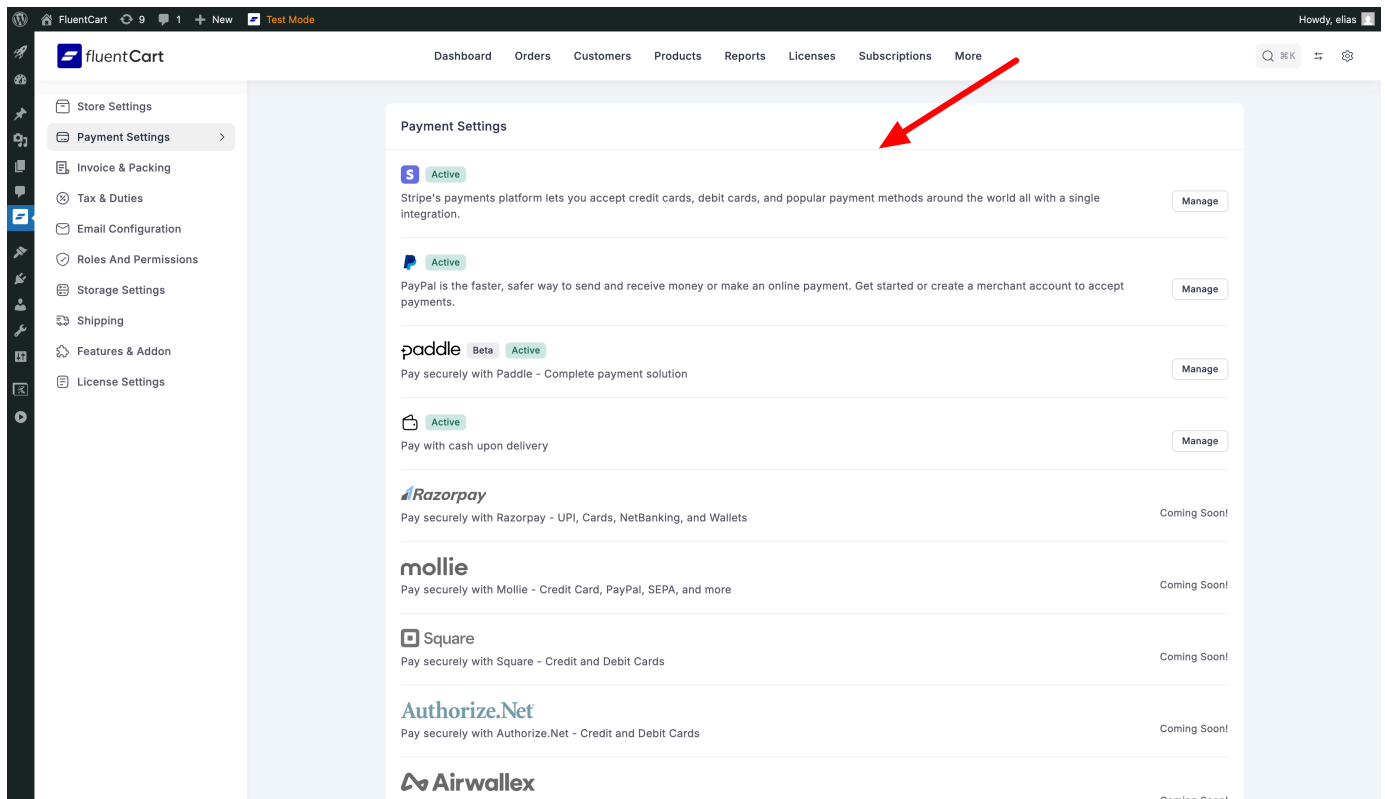
```
'handle' => 'your-gateway-external-lib',
'src' => $gatewayLibUrl,
],
[
    'handle' => 'fluent-cart-your-gateway-checkout',
    'src' => plugin_dir_url(__FILE__) . 'assets/js/your-gateway-che
    'deps' => ['your-gateway-external-lib'],
    'version' => FLUENTCART_PLUGIN_VERSION
]
];
}
```

fields() method setup

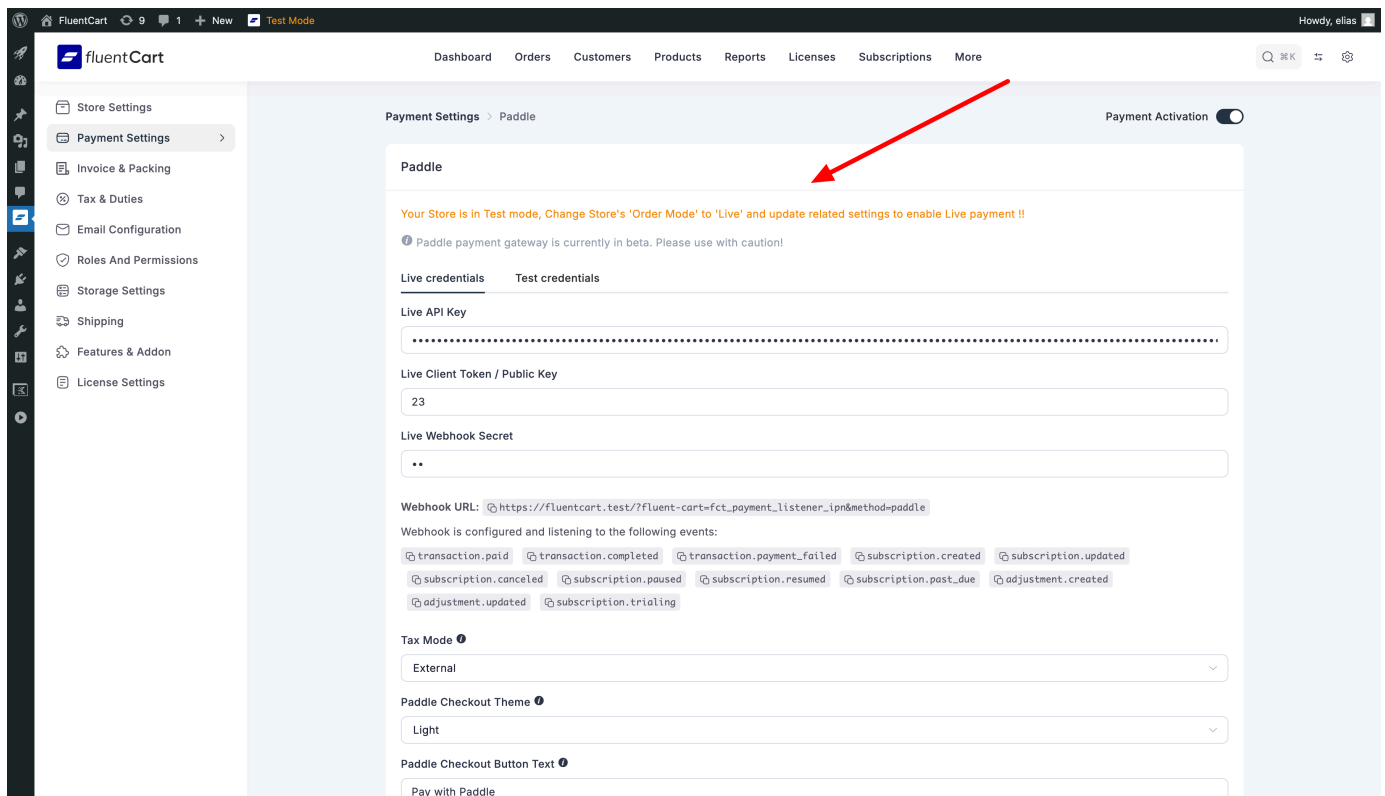
For a comprehensive guide on building gateway settings fields, see the detailed documentation: [Payment Gateway Settings Fields](#)

Now your gateway registration is done, you will see your gateway in the payment methods list in FluentCart admin dashboard. And if you follow the [Fields](#) guide and Configure your gateway settings and save with **Payment activation** on, you will see the gateway in the payment methods list in FluentCart checkout page.

Payment methods list in FluentCart admin dashboard



Payment method settings configuration page



Active payment methods displayed in checkout page

Payment

Card

Card

US bank account

Cash App Pay

Google Pay

pay

Card number

1234 1234 1234 1234

VISA

MM / YY

Security code

CVC

Country

Bangladesh

By providing your card information, you allow Elias to charge your card for future payments in accordance with their terms.

PayPal

Cash

paddle Paddle

Place Order

Start taking payments with your gateway

Step 4: Create an API Handler

Create a class for API communications with your payment provider:

```
<?php
namespace YourPlugin\PaymentMethods\YourGateway;

class API
```

php


```

{
    private $settings;
    private $baseUrl;

    public function __construct(YourGatewaySettings $settings)
    {
        $this->settings = $settings;
        $this->baseUrl = $settings->isTestMode()
            ? 'https://api-test.yourgateway.com/v1'
            : 'https://api.yourgateway.com/v1';
    }

    public function makeRequest($method, $endpoint, $data = [])
    {
        $args = [
            'method' => $method,
            'headers' => [
                'Authorization' => 'Bearer ' . $this->settings->getApiKey(),
                'Content-Type' => 'application/json',
            ],
            'timeout' => 30
        ];

        if (!empty($data) && in_array($method, ['POST', 'PUT'])) {
            $args['body'] = json_encode($data);
        }

        $response = wp_remote_request($this->baseUrl . $endpoint, $args);

        if (is_wp_error($response)) {
            return [
                'success' => false,
                'message' => $response->get_error_message()
            ];
        }

        $body = wp_remote_retrieve_body($response);
        $data = json_decode($body, true);
        $code = wp_remote_retrieve_response_code($response);

        if ($code >= 400) {

```

```
        return [
            'success' => false,
            'message' => $data['error'] ?? 'API request failed'
        ];
    }

    return [
        'success' => true,
        'data' => $data
    ];
}

public function createPayment($paymentData)
{
    return $this->makeRequest('POST', '/payments', $paymentData);
}

public function createSubscription($subscriptionData)
{
    return $this->makeRequest('POST', '/subscriptions', $subscriptionData);
}

public function refundPayment($paymentId, $amount, $reason = '')
{
    return $this->makeRequest('POST', "/payments/{\$paymentId}/refund", [
        'amount' => $amount,
        'reason' => $reason
    ]);
}
}
```

Step 5: Payment processing

<?php

php

#YourGateway.php

```

public function makePaymentFromPaymentInstance(PaymentInstance $paymentInstance)
{
    $order = $paymentInstance->order;

    if ($paymentInstance->subscription) {
        return (new Processor())->handleSubscriptionPayment($paymentInstance);
    }

    // check if this is a subscription payment
    if ($paymentInstance->subscription) {
        return $this->handleSubscriptionPayment($paymentInstance);
    }

    // This is a regular one-time payment
    return $this->handleOneTimePayment($paymentInstance);
}

// Example of one-time payment processing

// option 1: make payment with hosted(redirect) checkout
private function handleOneTimePayment(PaymentInstance $paymentInstance)
{
    $order = $paymentInstance->order;
    $transaction = $paymentInstance->transaction;

    $api = new API($this->settings);

    // Prepare payment data for the API
    $paymentData = [
        ....
    ];

    // Create payment in your gateway
    $result = $api->createPayment($paymentData);

    if ($result['success']) {
        // Store payment ID for later reference
        $transaction->update([
            ...
        ]);
    }
}

```

```

// Return redirect URL (for redirect-based gateways)
return [
    'redirect_to' => $result['data']['checkout_url'],
    'status'      => 'success',
    'message'     => __('Order has been placed successfully', 'fluent-c
];
}
}

```

```

// option 2: make payment onsite
private function handleOneTimePayment(PaymentInstance $paymentInstance)
{
    $order = $paymentInstance->order;
    $transaction = $paymentInstance->transaction;

    $api = new API($this->settings);

    // Prepare payment data for the API
    $paymentData = [
        ....
    ];

    // Create payment in your gateway
    $result = $api->createPayment($paymentData);

    if ($result['success']) {
        // Store payment ID for later reference
        $transaction->update([
            ...
        ]);

        // Return success response with custom action to redirect to your own y
        return [
            'nextAction'      => 'your_gateway',
            'actionName'      => 'custom',
            'status'          => 'success',
            'message' => __('Order has been placed successfully', 'fluent-cart'
            'response' => $result,
        ];
    }
}

```

```

    }
}

// ... subscription is similar to one-time payment

```

Step 6: Confirm Payment

Payment confirmation can be done in two ways:

Ajax call From you your-gateway-checkout.js (onsite payment)

php

```

<?php

#YourGateway.php

// init ajax handler in boot/constructor
public function boot()
{
    ....
    add_action('wp_ajax_fluent_cart_confirm_your_gateway_payment', [$this, 'confirmPayment']);
    add_action('wp_ajax_nopriv_fluent_cart_confirm_your_gateway_payment', [$this, 'confirmPayment']);
}

// Example
public function confirmPayment()
{
    // Get data from request
    $transactionId = sanitize_text_field($_REQUEST['transaction_id'] ?? '');
    $paymentId = sanitize_text_field($_REQUEST['payment_id'] ?? '');

    // Find the transaction by UUID (ref_id)
    $transaction = OrderTransaction::query()->where('uuid', $transactionId)->first();

    if (!$transaction) {
        wp_send_json([

```

```
        'message' => 'Transaction ID is required to confirm the payment.',
        'status' => 'failed'
    ], 400);
}

// Check if already processed
if ($transaction->status === Status::TRANSACTION_SUCCEEDED) {
    wp_send_json([
        'redirect_url' => $transaction->getReceiptPageUrl(),
        'order' => [
            'uuid' => $transaction->order->uuid,
        ],
        'message' => __('Payment already confirmed.', 'fluent-cart'),
        'status' => 'success'
    ], 200);
}

// Verify payment with gateway API
// $paymentStatus = YourGatewayAPI::verifyPayment($paymentId);

// Update transaction and order status
$transaction->fill([
    'status' => Status::TRANSACTION_SUCCEEDED,
    'vendor_charge_id' => $paymentId,
    // Add other fields as needed
]);
$transaction->save();

// Update order status
$order = Order::query()->find($transaction->order_id);
(new StatusHelper($order))->syncOrderStatuses($transaction);

// Send success response
wp_send_json([
    'redirect_url' => $transaction->getReceiptPageUrl(),
    'order' => [
        'uuid' => $transaction->order->uuid,
    ],
    'message' => __('Payment confirmed successfully.', 'fluent-cart'),
    'status' => 'success'
```

```
    ],  
  
}
```

With IPN/Webhooks (Hosted payment)

php

```
<?php  
  
// YourGateway.php  
  
// init ipn handler in boot/constructor  
public function boot()  
{  
    ....  
    add_action('fluent_cart/payments/your_gateway/webhook_payment_completed', [  
    ....  
}]  
  
public function handleIPN(): void  
{  
    // Process the webhook  
    $this->processWebhookEvent($data);  
}  
  
// Example of webhook processing  
public function processWebhookEvent($data)  
{  
    $eventType = $data['event'] ?? '';  
    $eventTypeFormatted = str_replace('.', '_', $eventType);  
  
    // Fire specific event handler  
    if (has_action('fluent_cart/payments/your_gateway/webhook_' . $eventTypeFormatted))  
        do_action('fluent_cart/payments/your_gateway/webhook_' . $eventTypeFormatted,  
            'data' => $data,  
            'raw' => $rawPayload,  
            'order' => $order  
        );  
}  
}
```

```
// Example of payment confirmation
public function handlePaymentCompleted($data)
{
    $paymentId = $data['payment']['id'] ?? '';

    // Find the transaction
    $transaction = OrderTransaction::query()->where('uuid', $transactionId)->fi

    if (!$transaction) {
        return;
    }

    // Check if already processed
    if ($transaction->status === Status::TRANSACTION_SUCCEEDED) {
        return;
    }

    // Get transaction details from YourGateway
    $yourGatewayTransaction = API::getYourGatewayObject("transactions/{$payment

    if (is_wp_error($yourGatewayTransaction)) {
        return;
    }

    $data = Arr::get($yourGatewayTransaction, 'data');
    $transactionStatus = Arr::get($data, 'status');

    // Check if payment is completed
    if ($transactionStatus !== 'paid' && $transactionStatus !== 'completed') {
        return;
    }

    // Update transaction and order status
    $transaction->fill([
        'status' => Status::TRANSACTION_SUCCEEDED,
        'vendor_charge_id' => $paymentId,
        // Add other fields as needed
    ]);
    $transaction->save();
}
```



```
// Update order status
$order = Order::query()->find($transaction->order_id);
(new StatusHelper($order))->syncOrderStatuses($transaction);
}
```

Step 7: (optional) Create your-gateway-checkout.js

Create custom JavaScript file to handle (onsite) payment checkout

Example of onsite payment checkout with custom checkout button

javascript

```
// File: assets/js/your-gateway-checkout.js
class YourGatewayCheckout {
  constructor(form, orderHandler, response, paymentLoader) {
    this.form = form;
    this.orderHandler = orderHandler;
    this.response = response;
    this.paymentLoader = paymentLoader;
    this.paymentArgs = response?.payment_args || {};
  }

  init() {
    // Find the payment container
    const paymentContainer = document.querySelector('.fluent-cart-checkout_');
    if (!paymentContainer) {
      console.error('Payment container not found');
      return;
    }

    // Create payment button
    this.createPaymentButton(paymentContainer);

    // Initialize gateway SDK (if applicable)
    this.initGatewaySDK();
  }

  createPaymentButton(container) {
```

```
// Clear container
container.innerHTML = '';

// Create button
const button = document.createElement('button');
button.type = 'button';
button.className = 'your-gateway-checkout-button';
button.textContent = 'Pay with Your Gateway';
button.style.cssText = `
    width: 100%;
    padding: 12px;
    background: #007bff;
    color: #fff;
    border: none;
    border-radius: 4px;
    font-size: 16px;
    cursor: pointer;
`;

// Add click event
button.addEventListener('click', () => {
    this.handlePaymentButtonClick();
});

// Add to container
container.appendChild(button);
}

initGatewaySDK() {
    // Initialize your gateway's SDK if required
    if (window.YourGatewaySDK) {
        window.YourGatewaySDK.initialize({
            publicKey: this.paymentArgs.public_key,
            environment: this.paymentArgs.mode
        });
    }
}

async handlePaymentButtonClick() {
    try {
        this.paymentLoader?.changeLoaderStatus('processing');
    }
}
```

```

// Create order in FluentCart first
const orderResponse = await this.orderHandler.createOrder();

if (!orderResponse?.success) {
    throw new Error('Failed to create order');
}

// For redirect-based gateways
if (orderResponse.redirect_url) {
    window.location.href = orderResponse.redirect_url;
    return;
}

// For JS-based gateways, open checkout modal
if (window.YourGatewaySDK) {
    const result = await window.YourGatewaySDK.openCheckout({
        amount: this.response.amount,
        currency: this.response.currency,
        orderId: orderResponse.order_id,
        onSuccess: (data) => {
            // Confirm payment on your server
            this.confirmPayment(data, orderResponse);
        },
        onCancel: () => {
            this.paymentLoader?.hideLoader();
        }
    });
}
} catch (error) {
    this.paymentLoader?.changeLoaderStatus('Error: ' + error.message);
    this.paymentLoader?.hideLoader();
}
}

async confirmPayment(gatewayData, orderData) {
    try {
        // Confirm payment on your server
        const confirmResponse = await fetch(fluentCartData.ajax_url, {
            method: 'POST',
            headers: {"Content-Type": "application/json"},

```

```

        credentials: 'same-origin',
        body: JSON.stringify({
            action: 'fluent_cart_confirm_your_gateway_payment',
            transaction_id: orderData.transaction_id,
            payment_id: gatewayData.paymentId
        })
    });

    const confirmation = await confirmResponse.json();

    if (confirmation.success && confirmation.redirect_url) {
        window.location.href = confirmation.redirect_url;
    } else {
        throw new Error(confirmation.message || 'Payment confirmation f
    }
} catch (error) {
    this.paymentLoader?.changeLoaderStatus('Error: ' + error.message);
    this.paymentLoader?.hideLoader();
}
}

}

// Initialize when FluentCart triggers the event
window.addEventListener("fluent_cart_load_payments_your_gateway", function (e)
    fetch(e.detail.paymentInfoUrl, {
        method: "POST",
        headers: {
            "Content-Type": "application/json"
        },
        credentials: 'include'
    })
    .then(response => response.json())
    .then(data => {
        new YourGatewayCheckout(
            e.detail.form,
            e.detail.orderHandler,
            data,
            e.detail.paymentLoader
        ).init();
    })
    .catch(error => {

```

```
        console.error('Error initializing gateway:', error);
    });
});
```

Key FluentCart Services to Use

FluentCart provides these services to make gateway development easier:

StatusHelper for Order Updates

```
// Update order status and fire all necessary hooks
(new \FluentCart\App\Helpers\StatusHelper($order))->syncOrderStatuses($transaction);

// This automatically fires:
// - fluent_cart/order_paid (for successful payments)
// - fluent_cart/order_failed (for failed payments)
// - fluent_cart/order_status_updated
$data[
    'order'      => $this->order,
    'customer'   => $this->customer ?? null,
    'transaction' => $this->transaction ?? null
];
```

php

Refund Service for Handling Refunds

```
// Use Refund service for all refund operations
\FluentCart\App\Services\Payments\Refund::createOrRecordRefund([
    'vendor_charge_id' => $refundId,
    'payment_method' => 'your_gateway',
    'status' => 'refunded',
    'total' => $refundAmount,
], $parentTransaction);
```

php

```
// This automatically handles:  
// - Creating refund transaction  
// - Updating order status  
// - Firing all refund-related hooks
```

Subscription Handling

For subscription renewals, use the SubscriptionRenewal service:

```
php  
\FluentCart\App\Services\Subscription\SubscriptionRenewal::recordRenewalPayment  
    $subscription,  
    [  
        'amount' => $amount,  
        'transaction_id' => $paymentId,  
        'payment_method' => 'your_gateway',  
        'status' => 'completed'  
    ]  
);
```



Important Hooks

Key hooks to be aware of:

Payment Hooks (Handled by StatusHelper):

- fluent_cart/order_paid
- fluent_cart/order_failed
- fluent_cart/order_status_updated

```
php  
$data = array(  
    'order'      => $this->order,  
    'customer'   => $this->customer ?? null,
```

```
'transaction' => $this->transaction ?? null  
);
```

Subscription Hooks:

- `fluent_cart/subscription_created`
- `fluent_cart/subscription_activated`
- `fluent_cart/subscription_renewed`
- `fluent_cart/subscription_cancelled`

```
$data = array(  
    'subscription' => $this->subscription,  
    'order' => $this->order,  
    'customer' => $this->customer ?? [],  
);
```

php

Learn more about hooks in [FluentCart Hooks](#) documentation.

Testing Your Gateway

Install & Activate: Activate your plugin in WordPress

Configure: Go to FluentCart → Settings → Payment Methods

Test Payment: Test a one-time payment on the checkout page

Test Subscription: Test a subscription product if supported

Test Web Hook: Test web hook processing using a tool like RequestBin

Additional Resources

For more detailed examples, you can refer to:

- Built-in payment methods in FluentCart (Stripe, PayPal)
- [Paddle Gateway Implementation](#) for a complete, real-world example

FluentCart developer documentation

Copyright © 2025 FluentCart